# 📚 Lecture 3: Vectorless RAG & PageIndex

## The New Paradigm That Challenges Vector Databases

## Part of the RAG Mastery Series

---

## 🎯 What You Will Learn Today

1. Why people are questioning vector databases
2. What is Vectorless RAG?
3. PageIndex — the new approach to document retrieval
4. BM25 and its modern variants
5. When to use vectors vs vectorless approaches
6. Hybrid approaches — the real answer

---

# 1. The Vector Database Debate

## The Traditional View (What You've Been Taught)

"Always use vector databases for RAG. Convert text to embeddings, store in vector DB, search by similarity."

## The Emerging Challenge (2024-2026)

Some researchers and companies are saying: **"Vector databases are overkill for many RAG use cases."**

**Why the doubt?**

| Problem with Vectors | Explanation |
|---|---|
| **Embedding quality varies** | Bad embeddings = bad retrieval, no matter how good the DB is |
| **Chunking destroys context** | Splitting documents loses relationships between sections |

| | |
|---|---|
| **Semantic search isn't always better** | For exact terms, names, codes — keyword search wins |
| **Cost** | Embedding millions of chunks costs money (API) or GPU (local) |
| **Complexity** | Vector DBs add infrastructure complexity |
| **"Vector search dilution"** | As you add more documents, vector search quality degrades |

## The Research Evidence

Recent papers (2025-2026) have shown:

- **"When More Documents Hurt RAG"** — Vector search quality drops as corpus grows
- **"One Polluted Page Is Enough"** — A single bad document can poison vector retrieval
- Simple BM25 + reranking often matches or beats pure vector search on many benchmarks

---

# 2. What is Vectorless RAG?

**Vectorless RAG = RAG without vector databases. Using traditional text retrieval + LLMs.**

## The Vectorless RAG Pipeline

```
Traditional RAG:
  Documents → Chunk → Embed → Vector DB → Vector Search → LLM

Vectorless RAG:
  Documents → Index (text) → BM25/TF-IDF Search → Rerank → LLM
```

## Key Techniques in Vectorless RAG

### BM25 (Best Matching 25)

The classic keyword search algorithm. Still very powerful!

**How BM25 works:**

1. Count how many times each word appears in a document (term frequency)

2. Rarer words across all documents get higher weight (inverse document frequency)
3. Normalize for document length
4. Score each document for the query

```
Query: "machine learning algorithms"

Document 1: "Machine learning algorithms are used in AI"
  → "machine" appears 1 time, "learning" 1 time, "algorithms" 1 time
  → All words are present → HIGH SCORE

Document 2: "Deep learning is a subset of machine learning"
  → "machine" appears 1 time, "learning" 2 times, "algorithms" 0 times
  → Missing "algorithms" → MEDIUM SCORE

Document 3: "The weather is nice today"
  → No matching words → ZERO SCORE
```

**BM25 is surprisingly good** and often beats vector search for:

- Exact keyword matching
- Named entities (people, places, products)
- Code search
- Technical terms

### Modern BM25 Variants

- **BM25+**: Improved version with better length normalization
- **BM25F**: Handles multiple fields (title, body, tags separately)
- **BM25-ADA**: Uses learned term weights
- **SPLADE**: Sparse learned retrieval (neural BM25)

---

# 3. PageIndex — The New Approach

## What is PageIndex?

**PageIndex** is a retrieval approach that indexes **pages** (or large sections) instead of small chunks, and uses the LLM's ability to find relevant information within large contexts.

## The Core Idea

Instead of:

```
Document → 50 small chunks → 50 vectors → retrieve 3 chunks → LLM
```

Do this:

```
Document → 5 large pages → index page summaries → retrieve 1-2 full pages → LLM
```

## How PageIndex Works

### Step 1: Create Page-Level Index

```
Page 1: "Introduction to Machine Learning" → Summary: "Covers ML basics, types of learning"
Page 2: "Supervised Learning" → Summary: "Classification, regression, training process"
Page 3: "Unsupervised Learning" → Summary: "Clustering, dimensionality reduction"
Page 4: "Neural Networks" → Summary: "Perceptrons, backpropagation, deep learning"
Page 5: "Evaluation Metrics" → Summary: "Accuracy, precision, recall, F1 score"
```

### Step 2: Query-Time Retrieval

```
User: "How do I evaluate a classification model?"

1. Match query against page summaries (BM25 or simple matching)
2. Page 5 ("Evaluation Metrics") is the best match
3. Send the ENTIRE Page 5 to the LLM
4. LLM extracts the relevant information and answers
```

## Advantages of PageIndex

| Advantage | Explanation |
|---|---|
| **No chunking needed** | Preserves document structure and context |
| **No embeddings needed** | Saves cost and complexity |
| **Better for long documents** | Full page context is more informative |
| **LLMs are good at finding info** | Modern LLMs can scan large contexts efficiently |
| **Simpler pipeline** | Fewer components to maintain |

### When PageIndex Works Best

✅ Documents with clear page/section structure
✅ When you have a manageable number of pages (< 10,000)
✅ When context within a page is important
✅ When you want to avoid embedding costs
✅ When documents are well-organized

### When PageIndex Doesn't Work Well

❌ When you have millions of documents
❌ When you need very specific sentence-level retrieval
❌ When documents don't have clear structure
❌ When the LLM's context window is limited

---

# 4. The Sparse Retrieval Renaissance

## SPLADE (Sparse Lexical and Expansion)

SPLADE is a neural sparse retrieval model — it combines the best of keyword search and neural understanding.

**How SPLADE works:**

1. Uses a BERT-like model to predict which words are important
2. Creates a sparse vector (mostly zeros, few important words)
3. Searches using these sparse representations
4. Captures semantic meaning while staying sparse

```
Query: "machine learning"
SPLADE representation: {machine: 1.2, learning: 1.1, AI: 0.8, algorithm: 0.6, ...}
(Only ~50-100 non-zero entries out of 30,000 vocabulary)
```

**Why SPLADE is exciting:**

- As accurate as dense embeddings on many benchmarks
- Much faster to search (sparse operations)
- Interpretable (you can see which words mattered)
- No vector database needed (can use Elasticsearch)

## ColBERT (Contextualized Late Interaction)

ColBERT keeps token-level embeddings and does "late interaction" at query time.

```
Query tokens: [q1, q2, q3]     → Each has its own embedding
Document tokens: [d1, d2, ..., dn] → Each has its own embedding

Score = max_sim(q1, all_d) + max_sim(q2, all_d) + max_sim(q3, all_d)
```

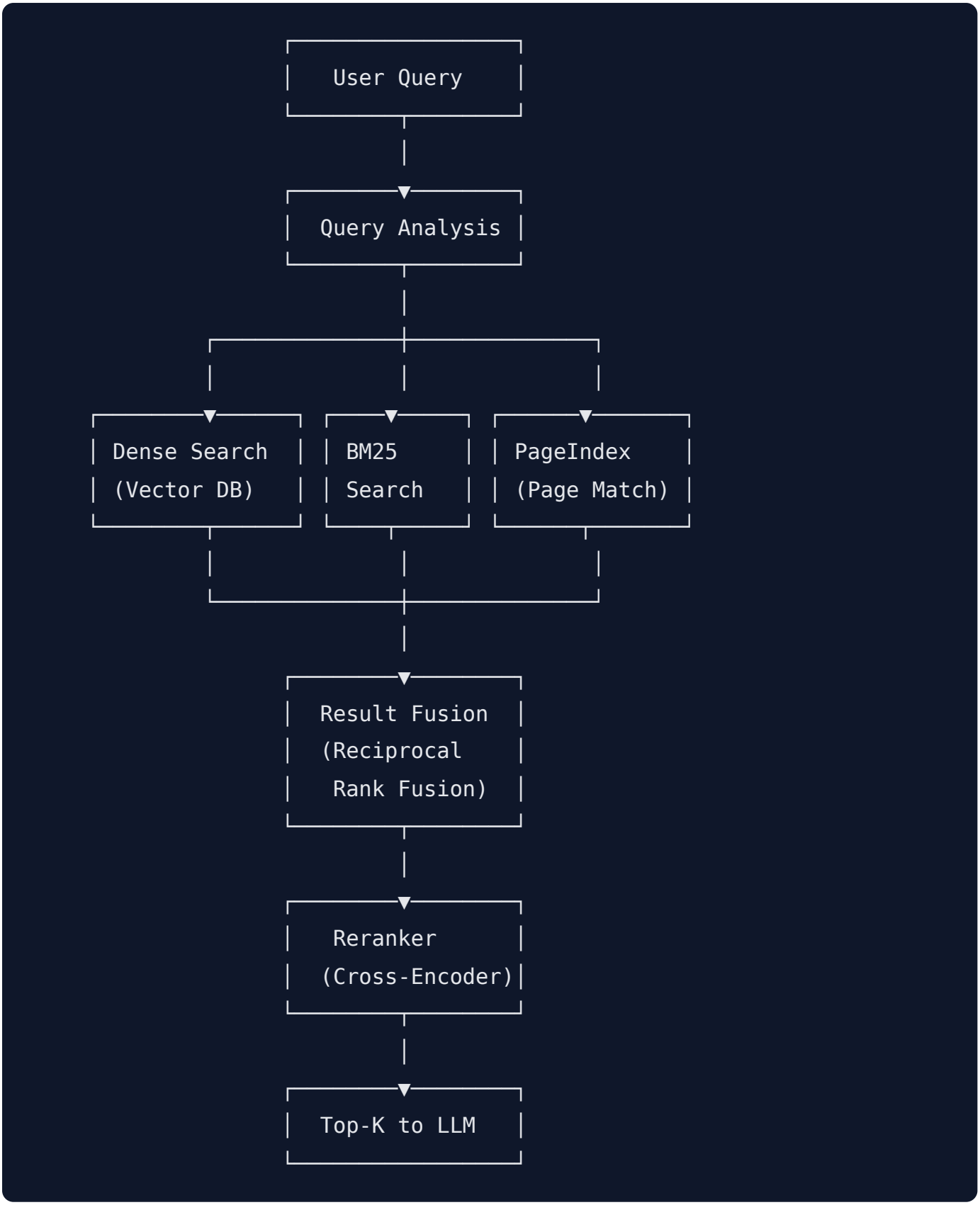
**Advantage**: Much more precise than single-vector retrieval
**Disadvantage**: More storage needed (stores per-token embeddings)

---

# 5. The Real Answer: Hybrid Retrieval

In 2025-2026, **the best RAG systems use hybrid retrieval** — combining multiple approaches.

## Hybrid Retrieval Architecture




## Reciprocal Rank Fusion (RRF)

A simple but effective way to combine results from multiple retrievers:

```
For each document d:
  RRF_score(d) = Σ 1/(k + rank_i(d))

Where:
  k = constant (usually 60)
  rank_i(d) = rank of document d in retriever i's results

Example:
  Doc_A: rank 1 in BM25, rank 3 in Vector
  RRF(Doc_A) = 1/(60+1) + 1/(60+3) = 0.0164 + 0.0159 = 0.0323

  Doc_B: rank 2 in BM25, rank 1 in Vector
  RRF(Doc_B) = 1/(60+2) + 1/(60+1) = 0.0161 + 0.0164 = 0.0325

  → Doc_B ranks higher overall!
```

---

# 6. Vectorless RAG with LLMs — The Extreme Approach

Some recent approaches skip retrieval entirely and let the LLM handle everything:

## Approach 1: Full Context

```
Put ALL documents in the prompt → Let LLM find the answer
```

- Works with small document sets
- No retrieval needed
- Limited by context window

## Approach 2: LLM-Powered Retrieval

```
LLM reads table of contents → Decides which pages to read →
LLM reads those pages → Generates answer
```

- Like how a human would use a book

- No vector DB needed
- Slower but can be more accurate

### Approach 3: Iterative Retrieval

```
LLM: "I need to find info about X"
→ Searches index → Reads results →
LLM: "Now I need more about Y"
→ Searches again → Reads results →
LLM: "Now I have enough to answer"
```

---

## 7. Quick Summary

```
Vectorless RAG: RAG without vector databases
PageIndex: Index pages instead of chunks, use LLM to find info within
pages
BM25: Still powerful for keyword search, often underrated
SPLADE: Neural sparse retrieval — best of both worlds
Hybrid: Combine dense + sparse + page-level for best results
RRF: Simple fusion method to combine multiple retrievers

Key Insight: There's no single best approach. Production RAG uses
multiple methods.
```

---

## 📝 Homework

1. Implement BM25 retrieval using `rank_bm25` library in Python
2. Compare BM25 vs vector search on the same dataset
3. Implement a simple PageIndex system (index page summaries, retrieve full pages)
4. Build a hybrid retriever that combines BM25 + vector search with RRF

---

*Next Lecture: Advanced RAG Patterns — Self-RAG, Corrective RAG, Agentic RAG, and Graph RAG*